

Uppgift 2

Avvikelser från MVC-idealet i det ursprungliga användargränssnittet

Problem 1

Controller hade för många ansvar. I originalet gjorde CarController allt:

- Höll listan av bilar
- Uppdaterade model state (move, gas, brake, etc.)
- Hanterade väggkollisioner
- Hanterade workshop-kollisioner
- Lyssnade på knappar
- Hanterade timerloop
- Tog fram positionsdata till DrawPanel

Detta bryter MVC eftersom Controller ska vara tunn, koordinera flödet, inte äga all logik.

Problem 2

View hade direkt beroende till modellen, t.ex:

- DrawPanel hade en ArrayList där den läste car.x, car.y, car.modelName
- Den bestämde vilken bilbild som skulle användas baserat på modelName

Den borde bara rita data, inte förstå modellens struktur.

Problem 3

Ingen tydlig Model-del, Det saknades:

- Ett koncept av "världen" eller "applikationsmodell"
- En central klass som ansvarade för bilarnas data och abstraktion
- Objekt för logik var utspritt

Man kunde med andra ord inte se ett renodlat "M" i MVC

Problem 4

Ingen data-abstraktion (DTO)

I den ursprungliga designen skickades modellobjekten direkt till DrawPanel.

Det innebar att View kunde läsa och använda modellens interna data, vilket skapade en onödigt stark koppling mellan Model och View. I en korrekt MVC-lösning borde View istället få:

- Modellens tillstånd
 - Enkla värden som bara beskriver det som ska ritas (t.ex. x-koordinat, y-koordinat och bilmodell)
 - Inte riktiga modellobjekt, eftersom det ger View för mycket insyn och ansvar
-

Problem 5

Krocklogik låg i Controller

- Väggekollision
- Workshopkollision

Detta är modellbeteende, inte Controller-beteende.

Vad borde ha gjort smartare / dummare / tunnare?

Model ska vara smart:

- Hålla tillstånd
- Validera regler (flak, turbo, ramp, hastighet)
- Sköta movement

View ska vara dum

- Bara visa data
- Inga modeller, ingen logik, inget game state
- Endast ta input från användaren

Controller ska vara tunn

- Ta emot inputs
 - Vidarebefordra till Model
 - Se till att View får rätt data (DTO)
 - Samordna system (movement, collision, workshop)
-

Vilka MVC-brister åtgärdade ni i del 3?

Brister som åtgärdades:

1. View är nu helt dum

- DrawPanel renderar endast List<CarDTO>
- Den känner inte längre modellen
- Den kan inte ändra Model
- Den har inget game-ansvar

2. Controller är tunnare, den delegerar till:

- MovementSystem
- CollisionSystem
- WorkshopCollisionSystem
- CarManager

Den gör inte längre all logik själv, mycket närmare MVC.

3. Löskoppling mellan View och Controller, vi införde:

- ICarController interface, view beror inte på konkret CarController.

4. Model kapslas i CarManager

- En central Model-datastruktur (CarManager)
- Gör Model mer sammanhållen

5. CarTransport delades upp i tre klasser

- Ramp
- CargoBay
- CarTransport

Detta följer SRP, bättre Model-del. Något vi borde gjort i tidigare labb redan.

6. Data går nu via DTO

- View får snapshots, ej Model-objekt
 - Ingen risk för felaktiga mutationer i View
-

Vilka MVC-brister åtgärdade ni inte i del 3?

1. Controller är fortfarande lite “för tjock”

Vi delar upp mycket i system (MovementSystem, CollisionSystem, osv.), men Controller innehåller fortfarande:

- Loops som gasar, bromsar och startar/stannar bilar
- Turbo av/på
- Flak upp/ner
- Timerns huvudloop

Det hade gått att fördela vissa av dessa saker ytterligare

3. CarManager är inte en fullständig Model-klass, CarManager håller bara:

- Listan av bilar
- Add/remove

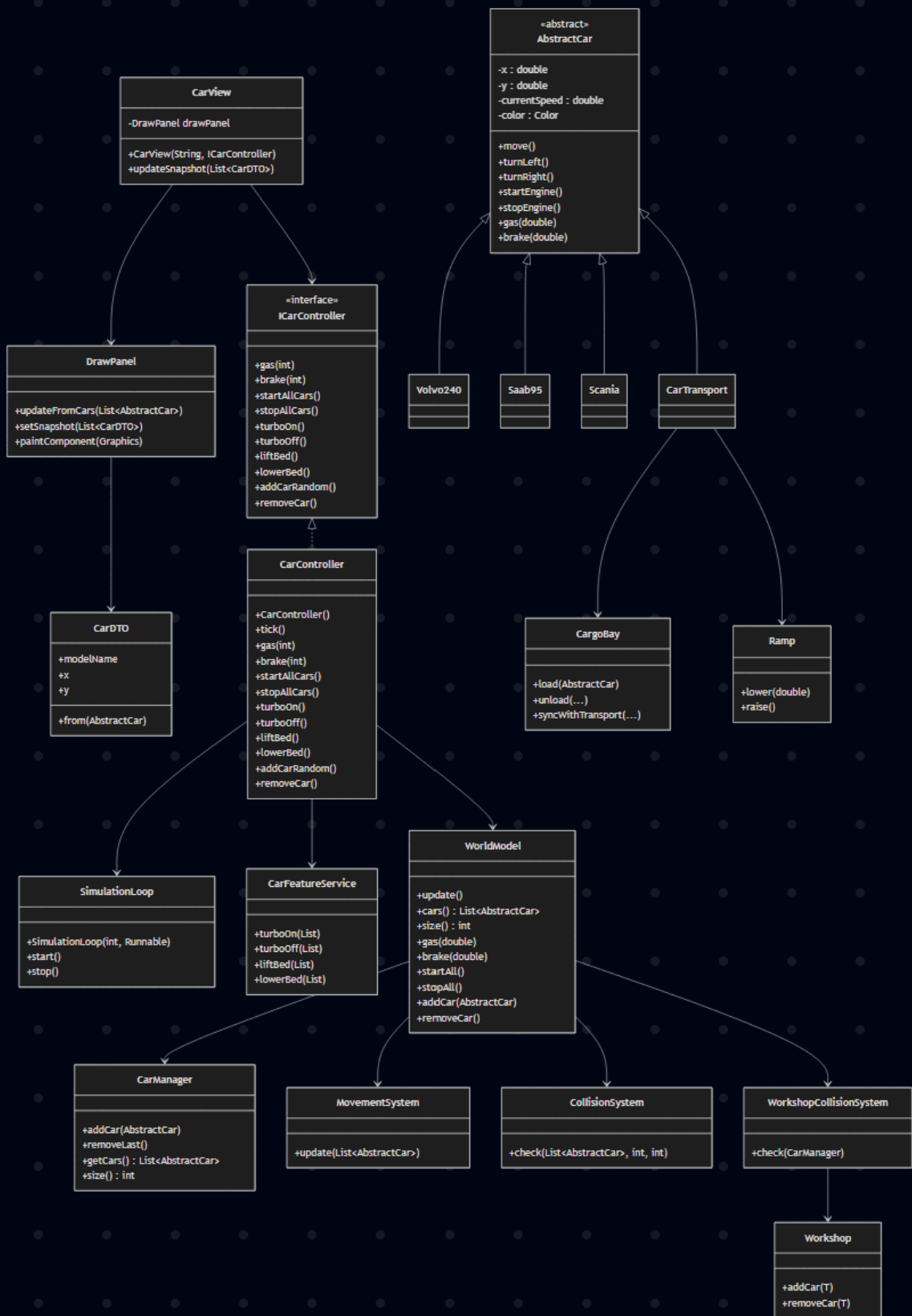
Men den ansvarar inte för:

- Movement
- Collisions
- Workshop-logik

Allt detta ligger fortfarande i separata system som Controller anropar.

Ett ännu renare MVC-upplägg hade haft en större Model som själv:

- Hanterar movement
- Hanterar kollisioner



UML följer MVC eftersom att:

- Lagerseparationen är tydlig:

View → Controller → Model är den enda beroenderiktningen, och Model är helt frikopplad från både Controller och View.

- Controller gör bara koordinering:

UML visar att CarController enbart anropar WorldModel, SimulationLoop och CarFeatureService, medan all rendering ligger i View och all logik i Model.

- View är helt passiv:

CarView och DrawPanel tar bara emot data och ritar; de innehåller ingen logik och känner inte till Model-klasser.

- Model är samlad och självständig:

WorldModel kapslar rörelse, kollisioner och workshop-logik, och all bilbeteende ligger i Model-klasserna (AbstractCar, State, Transport, Systems).

- DTO bryter kopplingen mellan lager:

UML visar att View får CarDTO i stället för Model-objekt, vilket håller datalager och presentationslager åtskilda.

Uppgift 3

Recap:

Observer innebär att en avsändare (Subject) automatiskt informerar en eller flera observatörer (Observers) när dess tillstånd förändras.

Factory Method: Ett mönster där ett objekt skapas genom en fabrikmetsod istället för att skapa det direkt med new.

State innebär att ett objekts beteende förändras beroende på ett "tillståndsobjekt".

Composite är att man kan behandla en grupp av objekt på samma sätt som ett enskilt objekt.

Används Observer i vår design idag?

Delvis, men inte fullt ut.

View (DrawPanel) uppdateras inte direkt av modellen.

I stället är det Controller som skapar snapshots och anropar setSnapshot() till repaint().

Detta är ett slags manuellt observer-beteende, men inte ett riktigt Observer-mönster eftersom Model inte notifierar något själv.

Vilket designproblem löste vi ändå?

- Undviker att View känner till Modellen direkt
- Säkerställer att GUI ritas om när Controller säger det
- Gör View dum (bra enligt MVC)

Kan designen förbättras med ett riktigt Observer-mönster?

Ja, vi skulle kunna:

- Låta CarManager vara Subject
- Låta DrawPanel vara Observer
- Controller triggar endast "tick", och Model sköter eventet

Vilka problem det skulle lösa:

- Controller blir ännu tunnare
 - View uppdateras automatiskt när Model ändras
 - Renare MVC där Model äger sina egna förändringar
-

Används Factory-Method redan?

Ja, vi skapade:

CarFactory.createVolvo(x, y)

CarFactory.createSaab(x, y)

CarFactory.createScania(x, y)

CarFactory.createRandom(x, y)

Controller behöver inte veta hur bilarna skapas.

Vilket problem löste Factory-Method för oss?

- Minimerade kopplingen mellan Controller och Model
- Möjliggjorde "Add Car"-funktionen utan att ändra några modellklasser
- Enkla utökningar (lägg till nya biltyper i framtiden)

Kan Factory Method användas mer?

Ja, workshop skulle kunna ta emot bilar via en factory för att producera olika typer av serviceobjekt

Används State i vår design idag?

Inte fullt ut men vi har “mini-state” via booleans::

- Saab95.turboOn (bool)
- Ramp.isDown() i CarTransport
- Scania.angle avgör om gas är tillåten

Dessa är tillstånd men inte implementerade som State-mönster-klasser.

Vilka problem löser vår nuvarande lösning?

- Enkelt att förstå
- Klar separation av regler (t.ex. ramp får inte vara nere när bilen kör)

Kan State förbättra designen?

Ja, vi skulle kunna skapa:

- MovingState: bilen är aktiv och kan röra sig, gasa, bromsa och bete sig normalt
- StoredState: bilen är lagrad (t.ex. på flaket eller i verkstaden) och kan därför inte röra sig eller accelerera

Vad det skulle lösa:

- Flyttar komplexa regler bort från modellklasserna, t.ex: StoredState ignorerar gas() och move()
- Minskar if-else och exceptions mitt i metoder
- Förbättrar testbarhet (enkla state-klasser)

Används Composite i vår design idag?

Ja, CarManager fungerar redan som en Composite:

- manager.getCars()
- manager.addCar(...)
- manager.removeCar(...)
- manager.removeLast()

Och Controller gör gruppoperationer:

- for (car : manager.getCars()) car.startEngine();

Vilket problem löste Composite i vår design?

- Gör det lätt att hantera flera bilar som en helhet
- Möjliggör “Start all cars”, “Stop all cars”, “Gas all cars”

Kan Composite användas mer?

Ja, vi kan lägga in grupp beteenden direkt i CarManager, t.ex.:

- `manager.startAll()`
- `manager.stopAll()`
- `manager.gasAll(amount)`

Det gör Controller ännu tunnare, vilket är bra i MVC.

Uppgift 5

Kan ni implementera Add/Remove utan att ändra existerande klassfiler?

Ja, genom att lägga logiken i Controller-lagret, skapa CarFactory för objektskapande och använda CarManager för att hantera bilistan.

Vilket designmönster är relevant?

Factory Method, eftersom vi skapar bilar via factory methods istället för att ändra befintliga bilklasser.

Composite används genom CarManager, eftersom vi kan behandla en grupp bilar som en enhet.